

Library Management System

A simple **Library Management System** developed using **Python**, **Tkinter**, and **SQL**.

The project provides a graphical user interface (GUI) to manage books, students/members, and library transactions efficiently.

Features

-  Add new books to the library
-  Search for books
-  Update book information
-  Delete books
-  Add and manage library members/students
-  Issue books
-  Return books
-  View available and issued books
-  Store and manage data using SQL database
-  User-friendly graphical interface using Tkinter

Technologies Used

Technology	Purpose
Python	Main programming language
Tkinter	Graphical User Interface (GUI)
SQL	Database management
SQL Connector	Connects Python application with SQL database

Project Structure

```
Library-Management-System/  
|  
├─ main.py           # Main Python application  
├─ database.py       # Database connection and queries  
├─ library.sql       # SQL database/tables  
├─ README.md         # Project documentation  
└─ screenshots/     # Project screenshots
```

The file names may be different depending on how your project is organized.

Requirements

Before running the project, make sure you have:

- Python 3.x

- MySQL / SQL database
- Tkinter
- Required Python SQL connector

Install SQL Connector

For MySQL, install the connector using:

```
pip install mysql-connector-python
```

Tkinter usually comes pre-installed with Python on Windows.

Database Setup

1. Install and open your SQL/MySQL server.
2. Create a database for the project.
3. Import or run the SQL commands provided in the project.
4. Update the database connection details in the Python code.

Example:

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="your_password",
    database="library"
)
```

Replace the username, password, and database name with your own details.

How to Run

1. Clone the repository

```
git clone YOUR_GITHUB_REPOSITORY_LINK
```

2. Open the project folder

```
cd Library-Management-System
```

3. Install the required library

```
pip install mysql-connector-python
```

4. Configure the database

Make sure your SQL/MySQL server is running and your database credentials are correctly configured in the Python files.

5. Run the application

```
python main.py
```

The Tkinter-based Library Management System will open.

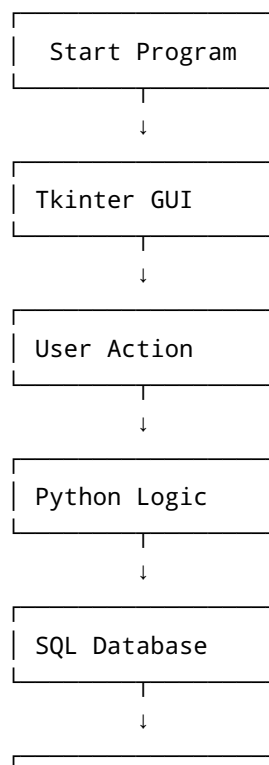
Working

The application uses **Tkinter** to provide the graphical interface. Users can interact with the system through buttons, forms, and tables.

Python processes the user's actions and communicates with the **SQL database** to:

- Insert new records
- Retrieve existing records
- Update information
- Delete records
- Manage book issue/return transactions

Basic Workflow



| Display Result |

Objective

The main objective of this project is to create a simple and efficient system for managing library operations while learning how to:

- Build GUI applications using Python Tkinter
- Connect Python with SQL databases
- Perform CRUD operations
- Work with database queries
- Develop a complete beginner-level Python project

Future Improvements

Some features that can be added in the future:

-  User login and authentication
-  Automatic due-date calculation
-  Fine calculation for late returns
-  Due-date notifications
-  Advanced reports and statistics
-  Advanced book filtering and searching
-  Separate admin and student accounts
-  Online/cloud database integration

Project Type

Academic / Beginner Python Project

This project was created to practice **Python GUI development, SQL database management, and CRUD operations.**

License

This project is created for educational purposes. You are free to modify and improve it for learning and academic use.